

Secure Coding & Code Quality Guidelines

Doc Ref: ACQO-DOC-001 | v1.0 | 2025

Autonomous Code Quality & Optimization Intelligence System

1. Overview

This document defines the secure coding standards, quality conventions, and engineering best practices adopted across all repositories managed under the Autonomous Code Quality & Optimization Intelligence System (ACQO). All development teams — Platform, Data, Security, and Frontend — must adhere to these guidelines to maintain consistency, security, and long-term maintainability across the codebase.

2. Coding Standards & Conventions

The following conventions apply across all supported languages and repositories:

Convention	Java (Microservices)	Python (Data)	JavaScript (Web)
Naming — Classes	PascalCase	PascalCase	PascalCase
Naming — Methods/Functions	camelCase	snake_case	camelCase
Naming — Constants	UPPER_SNAKE_CASE	UPPER_SNAKE_CASE	UPPER_SNAKE_CASE
Max Line Length	120 chars	88 chars (Black)	100 chars (Prettier)
Max Function Length	40 lines	30 lines	30 lines
File Max Lines	500 lines	400 lines	300 lines
Comment Style	Javadoc	Docstring (Google)	JSDoc

3. Secure Coding Practices

Security must be embedded at every stage of development. The following rules are mandatory for all repositories, with elevated scrutiny applied to authentication and payment modules:

- **Input Validation:** All user-supplied inputs must be validated at the boundary layer before processing. Never trust client-side validation alone.
- **Parameterised Queries:** Database queries must use prepared statements or ORM-level parameterisation. Raw string concatenation in SQL is prohibited.
- **Secrets Management:** API keys, passwords, and tokens must never be hardcoded. Use environment variables or a secrets vault (e.g. AWS Secrets Manager).
- **Authentication Tokens:** JWTs must be signed (RS256 preferred), have short expiry windows, and be validated server-side on every request.
- **Dependency Scanning:** All third-party dependencies must be scanned for known CVEs before merging. Tools: Dependabot, Snyk, or OWASP Dependency-Check.
- **Least Privilege:** Service accounts and IAM roles must be granted the minimum permissions necessary to perform their function.

4. Error Handling Best Practices

Scenario	Required Behaviour	Anti-Pattern to Avoid	Risk Level
Unhandled exception	Catch, log with context, return structured error	Silent catch block (catch {})	High
External API failure	Retry with backoff, surface timeout error	Infinite retry loop	High
Validation failure	Return 400 with field-level error messages	Returning 500 for bad input	Medium
Database error	Log query context, return 503, alert on-call	Exposing DB stack trace to user	High
Null/undefined value	Guard clause at entry, fail fast with clear message	Null pointer propagation	Medium

5. Logging Guidelines

Structured logging is mandatory. All log entries must conform to the following schema:

- **Required Fields:** timestamp (ISO 8601), log_level, service_name, trace_id, message, module.
- **Log Levels:** ERROR for exceptions and failures; WARN for recoverable degradation; INFO for lifecycle events; DEBUG for development only (must not reach production).
- **PII Prohibition:** Personal identifiable information (names, emails, card numbers) must never appear in logs. Mask or hash before logging.
- **Correlation IDs:** Every request must carry a trace_id propagated through all downstream calls for distributed tracing.

6. Input Validation Requirements

Input Type	Validation Rule	Enforcement Layer	Failure Response
API Request Body	Schema validation (JSON Schema / OpenAPI)	Gateway + Service	HTTP 400 + field errors
User-supplied strings	Length limits, whitelist pattern matching	Service layer	HTTP 400 + message
File uploads	MIME type check, size limit, virus scan	Inbound handler	HTTP 422
SQL inputs	Parameterised only — no raw interpolation	Repository layer	Reject + alert
Authentication tokens	Signature verification, expiry check, issuer validation	Auth middleware	HTTP 401

7. Dependency Management

Uncontrolled dependencies are one of the leading sources of security vulnerabilities:

- All dependencies must be pinned to exact versions in lockfiles (pom.xml, requirements.txt, package-lock.json).
- Major version upgrades require security review and regression testing sign-off.
- Transitive dependencies flagged with CVSS score ≥ 7.0 must be remediated within 7 days.
- Deprecated or unmaintained packages (>2 years since last release) must be replaced before production deployment.

Code Quality Enforcement Workflow

Code Commit

Static Analysis

Security Scan

Peer Review

CI Gate

Merge & Deploy

✓ Recommended Practices	✗ Prohibited Practices
✓ Validate all inputs at service boundaries	✗ Hardcode secrets or credentials in source files
✓ Use parameterised queries for all database access	✗ Use raw string interpolation in SQL queries
✓ Log with structured fields and correlation IDs	✗ Expose stack traces or DB errors to API consumers
✓ Pin all dependency versions in lockfiles	✗ Merge code with CVSS ≥ 7.0 CVEs without remediation

■ Note: All repository names, metrics, and code references are synthetic and created for educational simulation purposes only.